Lezione 14 - Il Framework JUnit

Introduzione a JUnit

Finora abbiamo visto come gestire dati in Java attraverso **Collection**, **Map** e relative implementazioni concrete come `HashMap` e `TreeMap`.
Abbiamo anche approfondito il concetto di **tipi primitivi e wrapper**, che è fondamentale per utilizzare correttamente le strutture generiche, come ad esempio:

- `int` → `Integer`
- `double` → `Double`
- `char` → `Character`

Questo perché molte classi generiche, comprese le collection e le mappe, possono lavorare solo con **oggetti reference**, e non con tipi primitivi.

In tutti questi casi, abbiamo definito classi, creato metodi, gestito chiavi, valori e strutture complesse.
Ma come possiamo essere sicuri che **il nostro codice funzioni correttamente**?
Come possiamo verificare che i metodi producano il risultato atteso, anche dopo modifiche successive?

Qui entra in gioco **JUnit**, il framework standard per il **testing unitario in Java**.

Cos'è JUnit

JUnit è **il framework di testing più diffuso nell'ecosistema Java.**
Fa parte della famiglia di framework **"xUnit"**, **progettati per scrivere e eseguire Unit Test automatici, cioè test che verificano il comportamento di singole unità di codice, come metodi o classi.**

L'uso di JUnit è ormai uno **standard de facto** per garantire la qualità del codice Java, perché permette di individuare errori precocemente, automatizzare il controllo dei comportamenti attesi e facilitare la manutenzione.

Breve storia

La storia di JUnit è strettamente legata all'evoluzione della pratica del testing:

- **1977**: **nasce il concetto di xUnit grazie a Kent Beck, pioniere dell'Extreme Programming, e Erich Gamma, uno degli autori del famoso libro *Design Patterns*.**
- **JUnit 3**: **per creare un test bisognava estendere una classe base, scrivendo i metodi di test al suo interno.**
- **JUnit 4 (2006)**: con l'introduzione delle annotazioni in Java 5, non era più necessario estendere una classe base.
  - Le annotazioni (`@Test` , `@Before` , ecc.) rendevano il codice più chiaro e modulare. Questa versione è rimasta lo standard per oltre un decennio.
- **JUnit 5 (2017)**: rivoluzione completa. Supporta **lambda expression** e **stream di Java 8**, sfrutta le annotazioni più moderne e introduce un'architettura modulare, più flessibile e potente.

Differenze principali tra JUnit 4 e JUnit 5

JUnit 5 non è un unico blocco monolitico come le versioni precedenti, ma è composto da **tre sotto-progetti** distinti:

1. **JUnit Platform**
   - È il motore che esegue i test, sia direttamente dall'IDE sia tramite strumenti di build come **Maven** o **Gradle**.
2. **JUnit Jupiter**
   - La nuova **API** **per scrivere i test** con le nuove annotazioni, lambda e stream.
   - È il cuore moderno di JUnit 5.
3. **JUnit Vintage**
   - Modulo di compatibilità che permette di eseguire **test scritti con JUnit 3 o 4** all'interno del nuovo ambiente, garantendo retrocompatibilità.

✓ **Questa evoluzione rende JUnit 5:**

- più modulare e flessibile
- compatibile con le vecchie versioni
- integrabile con strumenti moderni di sviluppo e build
- in grado di sfruttare pienamente le funzionalità avanzate di Java 8 e successive

Come funziona JUnit

Il funzionamento di JUnit può essere paragonato a quello di un **ispettore automatizzato** che verifica il corretto comportamento del nostro codice.

1️⃣ Preparazione del test

Il compito del **tester** (cioè dello sviluppatore) è preparare il **caso di test**:

- Si crea una **classe di test**, separata dalle classi di business, in cui si scrivono i metodi che vogliamo verificare.
- Per ogni metodo da testare si definiscono:
  - **i parametri di input**
  - **i risultati attesi, sia in caso di esito positivo sia in caso di errore o eccezione**

In pratica, il tester stabilisce **come dovrebbe comportarsi il codice** in ogni situazione prevista.

2️⃣ Esecuzione e valutazione

Una volta preparati i test, JUnit si occupa di:

1. **Preparare l'ambiente**
   - Creare eventuali oggetti necessari
   - Eseguire i metodi di setup definiti con annotazioni come `@BeforeEach`
2. **Eseguire i test**
   - Far partire i metodi di test, che a loro volta invocano i metodi delle classi da verificare
3. **Valutare i risultati**
   - Confrontare i risultati ottenuti con quelli attesi
   - Segnalare eventuali discrepanze, indicando esattamente quale test è fallito e perché

In questo modo JUnit permette di **automatizzare completamente la verifica del codice**, riducendo al minimo l'errore umano e aumentando la **fiducia nella correttezza delle classi implementate**.

Esempio pratico: test della classe `Calcolatrice`

Per capire concretamente come funziona JUnit, consideriamo una **classe semplice di business**: la `Calcolatrice`.

Questa classe espone alcuni metodi matematici di base, come `somma`, `divisione` e `potenzaPositiva`.

Vogliamo eseguire una serie di test per verificare che ciascun metodo produca il risultato corretto o, nei casi previsti, lanci l'eccezione giusta con il messaggio corretto.

1️⃣ Definizione dei casi di test

Alcuni esempi concreti:

Metodo	Input	Risultato atteso
`somma(1,1)`	1,1	2
`somma(-1,1)`	-1,1	0
`potenzaPositiva(2,3)`	2,3	8
`potenzaPositiva(-10,2)`	-10,2	eccezione con messaggio: "Numeri non positivi"
`divisione(10,4)`	10,4	2.5
`divisione(10,0)`	10,0	eccezione con messaggio: "errore divisione per zero"

Questi casi coprono sia **esiti corretti** sia **situazioni eccezionali**, in modo da testare la robustezza del codice.

2️⃣ Realizzazione del progetto in Eclipse con Maven

Per gestire facilmente le dipendenze necessarie a JUnit, conviene creare un **progetto Maven**:

1. **Creazione del progetto**
   - Group Id: `Testing`
   - Artifact Id: `Testing`
2. **Aggiunta della dipendenza JUnit nel** `pom.xml`

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.14.2</version>
```

```
  <scope>test</scope>
</dependency>
```

> ⚠️ ⚠️ **Lo `scope` test indica che la libreria JUnit sarà utilizzabile solo dalle classi che si trovano sotto `src/test/java`, mentre il codice di business sotto `src/main/java` resta indipendente.**

4. **Creazione della classe di business**
   - Sotto `src/main/java` creiamo la classe `Calcolatrice` con tutti i metodi da testare.
5. **Creazione della classe di test**
   - Sotto `src/test/java` creiamo la classe di test, ad esempio `FirstTest`, dove inseriremo i metodi annotati con `@Test`.

Il concetto di asserzione

In JUnit, il concetto chiave per valutare il successo di un test è **l'asserzione**.

Un'**asserzione** è:

- **una condizione logica che il programmatore assume essere sempre vera in un determinato punto del codice.**
  In altre parole, è:
- **una affermazione sul comportamento atteso di un metodo o di una funzione.**

Con JUnit, le asserzioni permettono di stabilire **se un test ha avuto successo o è fallito**, confrontando il risultato effettivo del metodo con quello previsto.

Esempi pratici di asserzioni

Alcuni esempi concreti:

1. **Controllo di uguaglianza**
   "Affermo che la somma di `1 + 1` **è uguale a 2**."
2. **Controllo di verità di una condizione**
   "Affermo che **è vero** che la somma di `1 + 1` è 2."
3. **Controllo di eccezioni**
   "Affermo che **invocando la divisione per zero viene sollevata un'eccezione**."

In tutti questi casi, l'asserzione definisce il **comportamento atteso**, e JUnit verificherà automaticamente se questo comportamento corrisponde al risultato effettivo.

La classe `Assertions`

JUnit fornisce la classe `org.junit.jupiter.api.Assertions`, che **mette a disposizione metodi statici per definire le asserzioni dei test. Questi metodi modellano i vari casi di test e permettono di gestire sia valori primitivi, sia oggetti, sia eccezioni.**

I principali metodi sono:

1. `assertEquals(expected, actual)`
   - **Verifica che il valore effettivo sia uguale a quello atteso**
   - **Supporta anche overloading per tipi primitivi e oggetti**
2. `assertTrue(BooleanSupplier supplier)`
   - **Verifica che la condizione logica fornita sia vera**
3. `assertFalse(BooleanSupplier supplier)`
   - **Verifica che la condizione logica fornita sia falsa**
4. `assertThrows(Class<? extends Throwable> exceptionClass, Executable methodCall)`
   - **Verifica che l'esecuzione di un determinato metodo produca l'eccezione specificata**

Nota sulle interfacce funzionali

I parametri `BooleanSupplier` e `Executable` sono interfacce funzionali che permettono l'uso delle **lambda expression** in Java 8+, rendendo le asserzioni più concise e leggibili.
Ad esempio:

```
assertTrue(() -> calcolatrice.somma(1,1) == 2);
assertThrows(ArithmeticException.class, () ->
calcolatrice.divisione(10,0));
```

In questo modo, possiamo **modellare ogni comportamento atteso** direttamente all'interno della chiamata all'asserzione, senza creare variabili temporanee o blocchi aggiuntivi.

`Supplier` e `Executable`

In Java, molte interfacce funzionali standard **servono a rappresentare fornitori di valori o comportamenti specifici.**
Nel contesto di JUnit, le due interfacce funzionali più rilevanti sono `BooleanSupplier` e `Executable`.

`BooleanSupplier`

`BooleanSupplier` è un'estensione di `Supplier<T>`, l'interfaccia funzionale standard di Java 8 che rappresenta un **fornitore di valori**.

- Nel caso di `BooleanSupplier`, **il valore fornito è un boolean.**
- Ha un solo metodo astratto:

```
boolean getAsBoolean();
```

Grazie alle lambda, possiamo implementarlo in modo conciso:

```
() -> calcolatrice.somma(1, 1) == 2
```

- Qui la lambda **non prende parametri**, restituisce un booleano e rappresenta direttamente la condizione che vogliamo verificare con `assertTrue` o `assertFalse`.

In altre parole, `BooleanSupplier` permette di:

- **incapsulare una condizione logica come oggetto, pronta per essere valutata da JUnit.**

`Executable`

`Executable` è un'interfaccia funzionale specifica di JUnit:

- **serve a rappresentare un'operazione che può generare eccezioni, utile soprattutto per testare comportamenti eccezionali.**
- Il metodo astratto è:

```
void execute() throws Throwable;
```

La lambda relativa ha la forma:

```
() -> { /* codice che può lanciare eccezioni */ }
```

Ad esempio:

```
assertThrows(ArithmeticException.class, () ->
calcolatrice.divisione(10, 0));
```

- Caratteristiche principali:
  - **Non prende parametri**
  - **Non restituisce valori**
  - **Può sollevare eccezioni**

In pratica, `Executable` permette a JUnit di:

- **eseguire un blocco di codice e verificarne eventuali eccezioni, rendendo i test di errori chiari, concisi e sicuri.**

🔗 Collegamento con le asserzioni

Le due interfacce si integrano perfettamente con le **asserzioni di JUnit 5**:

Interfaccia	Metodo	Uso tipico
`BooleanSupplier`	`getAsBoolean()`	`assertTrue`, `assertFalse` per verificare condizioni logiche
`Executable`	`execute()`	`assertThrows` per verificare che un metodo lanci l'eccezione attesa

Grazie a queste interfacce funzionali e alle lambda expression, i test diventano:

- più **concisi**
- più **leggibili**
- completamente **automatizzabili**

Esempio pratico di test con JUnit: la classe `FirstTest`

Per mettere in pratica quanto visto finora, realizziamo una **classe di test** chiamata `FirstTest` per verificare i metodi della classe `Calcolatrice`.

1 Struttura della classe di test

La classe di test segue alcune regole fondamentali:

1. **Proprietà per la classe da testare**
   All'interno della classe, definiamo una proprietà che rappresenta l'oggetto della classe di business da testare e lo istanziamo:

```
Calcolatrice calcolatrice = new Calcolatrice();
```

2. **Metodi di test**
   Ogni caso di test è realizzato come un metodo `void` senza parametri, con un nome descrittivo a piacere.
   Il metodo è annotato con `@Test`, indicazione che segnala a JUnit che si tratta di un **caso di test**.

2️⃣ Test dei metodi semplici

Somma con `assertEquals`

```
@Test
void testSommaPositiva() {
    assertEquals(2, calcolatrice.somma(1,1));
    assertEquals(0, calcolatrice.somma(-1,1));
}
```

- `assertEquals(expected, actual)` :
  - **confronta il risultato atteso con quello reale.**
  - **Se la condizione non è rispettata, il test fallisce automaticamente.**

3️⃣ Test con `assertFalse`

È possibile verificare condizioni logiche negative:

```
@Test
void testSommaNonDiversa() {
    assertFalse(() -> calcolatrice.somma(1,1) != 2);
}
```

- **Qui stiamo affermando che è falso che `1 + 1` sia diverso da 2.**
- **In altre parole, possiamo usare asserzioni anche per condizioni negative, a seconda del comportamento che vogliamo verificare.**

4️⃣ Test di eccezioni con `assertThrows`

Per i metodi che possono generare eccezioni, come `divisione` o `potenzaPositiva` con numeri non validi:

```
@Test
void testDivisionePerZero() {
    Exception e = assertThrows(ArithmeticException.class,
        () -> calcolatrice.divisione(10, 0));
    assertEquals("errore divisione per zero", e.getMessage());
}
```

- `assertThrows(ExceptionClass.class, Executable)`:
  - **verifica che venga lanciata l'eccezione corretta.**
  - **Possiamo anche controllare il messaggio associato all'eccezione con un'ulteriore asserzione.**

Esecuzione dei test con JUnit

Dopo aver creato le classi di **business** (`Calcolatrice`) e di **test** (`FirstTest`), il passo successivo è **lanciare i test** tramite il framework di JUnit.

> 🖊 **Non è necessario creare un `main` method: JUnit gestisce autonomamente l'esecuzione dei test.**

Passaggi in Eclipse

1. **Selezionare la classe di test** (`FirstTest`)
2. **Eseguire** tramite il comando:

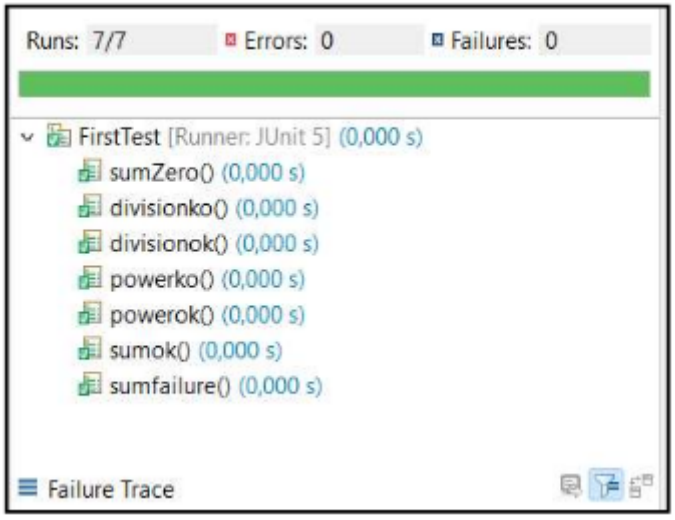
```
Run As > JUnit Test
```

L'IDE si occuperà di far partire automaticamente tutti i metodi annotati con `@Test` .

Interpretazione dell'output:
L'esecuzione produce un **report visivo** che segnala lo stato dei test:

- **Linea verde** → tutti i test sono stati eseguiti con successo
- **Numero di test eseguiti / totale** → indica quanti test sono stati lanciati
  - Descritti col nome in dettaglio
- **Numero di errori** → indicano problemi imprevisti, come eccezioni non gestite

- **Numero di fallimenti** → indicano che un test **ha prodotto un risultato diverso da quello atteso**



Esempio di failure

Supponiamo di scrivere un'asserzione sbagliata:

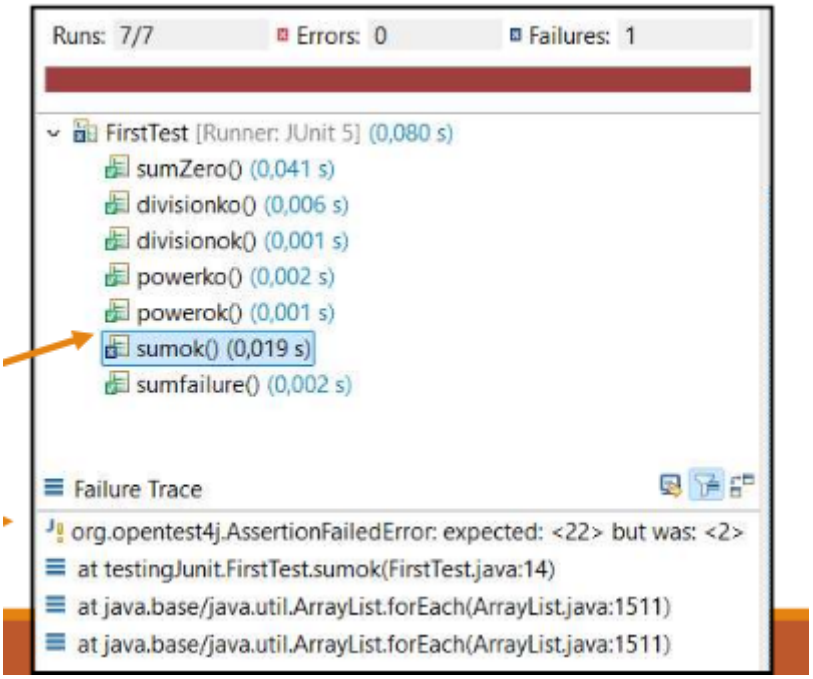
```
assertEquals(22, calcolatrice.somma(1,1));
```

Il report mostrerà:

- **Linea rossa** → indica un fallimento
- **7 test eseguiti su 7**
- **0 errori**
- **1 fallimento**, specificato nel dettaglio:

```
sumok()
expected: 22 but was: 2
```

Così, JUnit fornisce **informazioni precise sul test fallito**, rendendo semplice individuare e correggere l'errore.



In questo modo il ciclo di sviluppo con JUnit diventa **iterativo e sicuro**: puoi modificare il codice della `Calcolatrice` o aggiungere nuovi casi di test, eseguire nuovamente i test e verificare immediatamente se tutto funziona come previsto.

Diversi tipi di test

Il test che abbiamo appena realizzato sulla classe `Calcolatrice` è un **Unit Test**.

Un **Unit Test** è:

- **un test che verifica il comportamento di una singola unità di codice, tipicamente un metodo o l'insieme dei metodi di una sola classe, isolandoli dal resto dell'applicazione.**

L'obiettivo è controllare che quella specifica porzione di codice:

- produca il risultato corretto dato un certo input
- gestisca correttamente le eccezioni
- rispetti il comportamento atteso in ogni condizione prevista

Gli Unit Test sono:

- veloci da eseguire
- semplici da scrivere
- altamente automatizzabili
- fondamentali per garantire la correttezza della logica interna

Tuttavia, nello sviluppo software esistono anche altri livelli di test, che si collocano in una struttura comunemente rappresentata come una **piramide**.

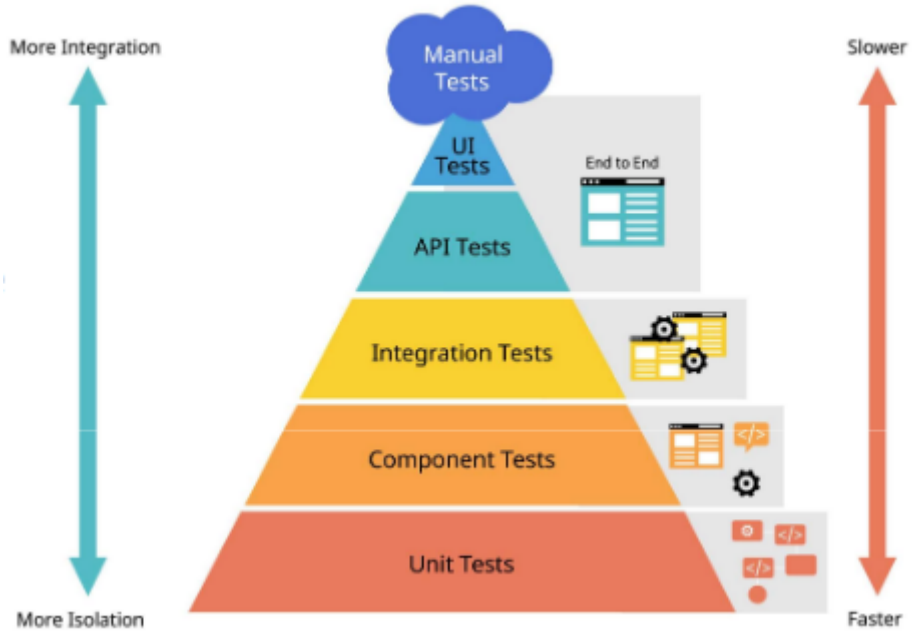
La piramide dei test

La cosiddetta *Test Pyramid* organizza i test in base a:

- **livello di astrazione**
- **costo di esecuzione**
- **complessità**
- **numero ideale di test**

Struttura della piramide

1. **Base – Unit Test**
   - Numerosi
   - Veloci
   - Economici da mantenere
   - Testano singole classi o metodi
2. **Livello intermedio – Integration Test**
   - Verificano l'interazione tra più componenti
   - Ad esempio: una classe che comunica con un database o con un servizio esterno
   - Più lenti e più complessi degli Unit Test
3. **Vertice – End-to-End (E2E) / System Test**
   - Testano l'intero sistema come farebbe un utente reale
   - Simulano scenari completi
   - Sono più costosi e più lenti
   - Devono essere meno numerosi




```
^piramidedeiTest
```

ⓘ Perché la forma a piramide?

La forma a piramide indica che:

- **Alla base devono esserci molti Unit Test**
- **Meno Integration Test**
- **Ancora meno End-to-End Test**

Questo perché:

- **Gli Unit Test sono rapidi e danno feedback immediato**
- **I test di livello superiore sono più costosi, fragili e complessi**

Una buona strategia di testing prevede quindi:

- **Forte copertura con Unit Test**
- **Test di integrazione mirati**
- **Pochi test end-to-end per verificare i flussi principali**

Facendo riferimento all'immagine della piramide dei test analizziamo i seguenti livelli partendo dal basso verso l'alto

Test di unità (Unit Test)

Gli **Unit Test** rappresentano il livello base della piramide dei test e costituiscono il primo strumento di verifica della qualità del codice.

Obiettivo

L'obiettivo principale è:

- **verificare la logica interna di un metodo o di una singola classe.**

Ad esempio:

- **verificare che il metodo** `somma` **restituisca il risultato corretto**
- **controllare che** `divisione` **gestisca correttamente il caso di divisione per zero**
- **assicurarsi che un metodo lanci un'eccezione quando riceve parametri non validi**

Il focus è sempre sull'unità minima di codice, senza considerare il comportamento dell'intero sistema.

Isolamento

Una caratteristica fondamentale degli Unit Test è l'**isolamento totale**.

Questo significa che il metodo testato non deve dipendere da:

- **database reali**
- file system
- **servizi esterni**
- API **remote**

Se la classe dipende da componenti esterni, questi vengono sostituiti con oggetti fittizi chiamati **Mock**, che simulano il comportamento delle dipendenze reali.

L'isolamento garantisce che:

- **eventuali errori dipendano esclusivamente dalla logica della classe testata**
- **il test sia deterministico (stesso input → stesso risultato)**
- **non ci siano interferenze esterne**

Test d'integrazione

I **Test d'integrazione** rappresentano il livello intermedio della piramide dei test. Se gli Unit Test verificano una singola unità di codice in isolamento, i test d'integrazione hanno un obiettivo diverso:

- **controllare che più componenti collaborino correttamente tra loro.**

Obiettivo

Lo scopo principale è:

- **verificare che due o più moduli si interfaccino e comunichino nel modo previsto.**

Ad esempio:

- **una classe di servizio che interagisce con un database reale**
- **un componente che invoca una API esterna**
- **un sistema che legge o scrive su file system**
- **più classi che collaborano tra loro all'interno di uno stesso flusso logico**

In questo caso non stiamo più testando la logica interna di un singolo metodo, **ma il comportamento combinato di più parti del sistema.**

Livello di isolamento

A differenza degli Unit Test, qui l'isolamento è **basso o nullo**.

- **Non vengono utilizzati mock per simulare le dipendenze**
- **Si lavora con componenti reali (database, servizi, file, ecc.)**

Questo consente di verificare:

- **che la configurazione sia corretta**
- **che le dipendenze siano integrate correttamente**
- **che il flusso tra i vari moduli non presenti errori**

Tuttavia, proprio perché si utilizzano componenti reali, il test diventa più complesso e potenzialmente meno stabile rispetto a un Unit Test.

Velocità

I Test d'integrazione sono **più lenti** rispetto agli Unit Test.

- **L'esecuzione può richiedere secondi o minuti**
- **Dipendono da risorse esterne (database, rete, I/O)**
- **Possono essere influenzati dall'ambiente di esecuzione**

Per questo motivo:

- **Devono essere meno numerosi rispetto agli Unit Test**
- **Vanno progettati con attenzione**
- **Servono principalmente a verificare i punti critici di integrazione**

Differenza concettuale rispetto agli Unit Test

Unit Test	Test d'integrazione
Verificano una singola unità	Verificano più moduli insieme
Isolamento totale (mock)	Dipendenze reali
Molto rapidi	Più lenti
Numerosi	Meno numerosi

In sintesi:

mentre gli Unit Test garantiscono

- **la correttezza della logica interna,**
  i Test d'integrazione assicurano
- **che le connessioni tra le parti del sistema funzionino correttamente.**

Test di componente

I **Test di componente** si collocano sopra gli Unit Test nella piramide, ma non arrivano ancora al livello dei Test d'integrazione completi.
Rappresentano un livello intermedio in cui non si verifica più il singolo metodo, **ma un modulo composto da più classi che collaborano tra loro.**

Obiettivo

Lo scopo è:

- **verificare che un componente applicativo, formato da diverse classi e responsabilità interne, funzioni correttamente nel suo insieme.**

Ad esempio:

- **un modulo** `GestioneOrdini` **composto da service, repository e validator**
- **un sottosistema che implementa una specifica funzionalità dell'applicazione**
- **un layer applicativo completo (es. service layer)**

In questo caso non si analizza più la singola unità di codice, ma si valuta il **comportamento complessivo del modulo**, considerandolo come un blocco funzionale coerente.

Livello di isolamento

L'isolamento è **medio o basso**.

- **Spesso si utilizzano classi reali**
- **Può essere coinvolto un database reale o semi-reale**
- **Si tende a isolare il modulo dal resto dell'applicazione, ma non dalle sue parti interne**

La logica interna del componente viene quindi testata in modo più realistico rispetto agli Unit Test, perché si verifica l'interazione tra le classi che lo compongono.

Velocità

I Test di componente sono:

- più lenti degli Unit Test
- generalmente più veloci dei Test d'integrazione completi

L'esecuzione può richiedere secondi, perché coinvolge più oggetti, più livelli logici e talvolta risorse esterne.

Differenza rispetto ai Test d'integrazione

La distinzione tra **Component Test** e **Integration Test** può essere sottile, perché entrambi si trovano sopra gli Unit Test nella piramide.

La differenza principale risiede nel **punto di vista**:

- Nel **Test di componente:**
  - **si considera un modulo come unità funzionale autonoma e lo si testa nel suo insieme.**
- Nel **Test d'integrazione:**
  - si verifica principalmente la comunicazione tra sistemi o moduli distinti, con particolare attenzione alle interfacce e ai confini tra componenti.

In altre parole:

- **Il Test di componente guarda all'interno di un modulo.**
- **Il Test d'integrazione guarda alle connessioni tra moduli o sistemi differenti.**

> **In sintesi:**
>
> i Test di componente rappresentano un livello di verifica più ampio rispetto agli Unit Test, ma ancora focalizzato su una parte ben definita del sistema, senza coinvolgere l'intera applicazione.

JUnit e la piramide di testing

Quindi abbiamo detto che JUnit è uno strumento **estremamente versatile** che può essere utilizzato in tutti i livelli della piramide di testing.

Tuttavia, **il suo ambito naturale rimane il Test di unità, dove esprime al massimo le sue potenzialità in termini di semplicità, velocità ed efficacia.**

JUnit nei diversi livelli della piramide

- **Unit Test**
  - È il contesto in cui JUnit viene utilizzato più frequentemente.
  - Permette di testare metodi e classi in isolamento, con esecuzione molto rapida e feedback immediato.
- **Component Test e Integration Test**
  - JUnit può essere impiegato anche per testare moduli composti da più classi o per verificare l'interazione tra componenti reali.
  - In questi casi il framework resta lo stesso, ma cambia l'approccio e il livello di isolamento.
- **Livelli superiori**

- Anche nei test più complessi JUnit può fungere da motore di esecuzione, integrandosi con altre tecnologie e strumenti.

JUnit e Mockito

Per mantenere l'isolamento tipico degli Unit Test, JUnit viene spesso utilizzato insieme a **Mockito**, un framework dedicato alla creazione di oggetti **Mock**.

I Mock sono oggetti fittizi che:

- **simulano il comportamento di altre classi**
- **sostituiscono dipendenze esterne (database, API, servizi remoti)**
- **permettono di controllare input e output in modo deterministico**

Ad esempio, se una classe dipende da un repository che interroga un database, con Mockito possiamo simulare il repository senza accedere realmente al database.

✓ Vantaggi dell'uso dei Mock

L'integrazione tra JUnit e Mockito consente di:

- mantenere il **massimo isolamento**
- ridurre drasticamente i tempi di esecuzione
- evitare dipendenze dall'ambiente esterno
- ottenere test ripetibili e affidabili

In questo modo si preservano le caratteristiche fondamentali degli Unit Test:

- velocità (millisecondi)
- determinismo
- semplicità di manutenzione

In sintesi:

JUnit rappresenta il **motore di esecuzione dei test**, mentre strumenti come Mockito permettono di controllare il livello di isolamento, adattando il test al livello desiderato della piramide.
Contents

Dettagli sulla classe di test

Una **classe di test** in JUnit segue alcune regole strutturali precise, che ne garantiscono il corretto funzionamento all'interno del framework.

Struttura della classe

Una classe di test:

- **Deve avere un solo costruttore, generalmente quello di default**
- **Può contenere diversi metodi dedicati ai casi di test**
- **I metodi possono anche non essere `public`, ma non devono essere `private`, altrimenti JUnit non riuscirà a invocarli**

> **JUnit istanzia automaticamente la classe di test e gestisce l'intero ciclo di vita degli oggetti, quindi non è necessario scrivere un metodo `main`.**

L'annotazione principale: `@Test`

L'annotazione fondamentale è:

```
@Test
```

Essa indica che il metodo annotato rappresenta un **singolo caso di test**.

Ogni metodo con `@Test`:

- **deve essere `void`**
- **non deve avere parametri**
- **deve contenere una o più asserzioni**

JUnit esegue automaticamente tutti i metodi annotati con `@Test` quando si lancia il test.

Metodi del ciclo di vita (Life Cycle Methods)

Oltre ai metodi di test, JUnit permette di definire metodi che vengono eseguiti in momenti specifici del ciclo di esecuzione.

Questi metodi servono per:

- **inizializzare risorse**

- **configurare l'ambiente**
- **rilasciare risorse dopo l'esecuzione**

Le principali annotazioni del ciclo di vita sono:

`@BeforeAll`

Il metodo annotato viene eseguito una sola volta prima di tutti i test.

Tipicamente viene utilizzato per:

- **inizializzazioni costose**
- **configurazioni globali**
- **apertura di connessioni condivise**

`@AfterAll`

Il metodo annotato viene eseguito **una sola volta dopo tutti i test**.

Serve per:

- **chiudere connessioni**
- **liberare risorse**
- **operazioni di cleanup globale**

`@BeforeEach`

Il metodo viene eseguito **prima di ogni metodo annotato con** `@Test`.

È utile per:

- **creare nuovi oggetti**
- **inizializzare variabili**
- **riportare lo stato a una condizione iniziale**

Questo garantisce che ogni test sia indipendente dagli altri.

`@AfterEach`

Il metodo viene eseguito **dopo ogni test**.

Serve per:

- **pulire lo stato**
- **rilasciare risorse specifiche del singolo test**

- **riportare il sistema in una condizione neutra**

> **Perché il ciclo di vita è importante?**
>
> Il ciclo di vita consente di:
>
> - mantenere i test indipendenti
> - evitare effetti collaterali tra test diversi
> - migliorare la leggibilità e l'organizzazione del codice
>
> In particolare, l'uso corretto di `@BeforeEach` è fondamentale per garantire che ogni test parta sempre dalle stesse condizioni iniziali.

Esempio pratico: gestione di un Database nei test

Supponiamo di voler testare un modulo che esegue **query su una tabella di un database**.

In questo scenario:

- **Prima dell'esecuzione di tutti i test vogliamo creare il database**
- **Dopo l'esecuzione di tutti i test vogliamo distruggere il database**

Questo è un caso tipico in cui utilizziamo i metodi del ciclo di vita `@BeforeAll` e `@AfterAll`.

Esempio 1/2 – Creazione e distruzione del DB

```
@BeforeAll
public static void createDB() {
    // creazione del DB
}

@AfterAll
public static void destroyDB() {
    // rimozione del DB
}
```

> **Caratteristiche importanti**
>
> - I metodi devono essere `void`
> - Non devono avere parametri

- Devono essere `static` (in JUnit 5, salvo configurazioni particolari)

Perché `static`?

Il motivo è legato al ciclo di vita della classe di test.

Per impostazione predefinita, JUnit crea **una nuova istanza della classe di test per ogni metodo annotato con `@Test`**.

Tuttavia:

- `@BeforeAll` e `@AfterAll` devono essere eseguiti **una sola volta**, prima e dopo tutti i test
- Per poterli invocare senza creare un'istanza specifica della classe, devono essere `static`

Un metodo `static` appartiene alla **classe**, non all'oggetto.
Quindi può essere eseguito senza istanziare la classe di test.

In sintesi:

- `@BeforeAll` e `@AfterAll` → livello di classe → `static`
- `@BeforeEach` e `@AfterEach` → livello di istanza → non `static`

Esempio 2/2 – Gestione della connessione per ogni test

Oltre alla creazione del database, vogliamo:

- Aprire una connessione prima di ogni test
- Chiudere la connessione dopo ogni test

Supponendo di avere una proprietà:

```
private Connection conn;
```

Possiamo scrivere:

```
@BeforeEach
public void openConnection() {
```

```
    try {
        this.conn = DriverManager.getConnection(url, username,
password);
    } catch (Exception e) {
        e.printStackTrace();
    }
}

@AfterEach
public void closeConnection() {
    try {
        this.conn.close();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Analisi del comportamento

- `@BeforeEach` viene eseguito **prima di ogni metodo di test**
  - Garantisce che ogni test abbia una connessione nuova
  - Evita dipendenze tra test
- `@AfterEach` viene eseguito **dopo ogni metodo di test**
  - Chiude la connessione
  - Libera le risorse
  - Mantiene l'ambiente pulito

📋 Differenza tra livello globale e livello per test

Annotazione	Frequenza	Uso tipico
`@BeforeAll`	1 volta prima di tutti i test	Creazione DB, configurazioni globali
`@AfterAll`	1 volta dopo tutti i test	Distruzione DB, cleanup globale
`@BeforeEach`	Prima di ogni test	Apertura connessioni, inizializzazione oggetti
`@AfterEach`	Dopo ogni test	Chiusura connessioni, reset stato

> ⚠ **Considerazione importante**
>
> Questo esempio mostra chiaramente perché:
>
> - Gli Unit Test puri tendono a evitare database reali (per mantenere alta velocità e isolamento)
> - Quando si lavora con DB reale si entra in un contesto più vicino ai **Test di componente o di integrazione**

Note sul ciclo di vita della classe di test

Perché `@BeforeAll` e `@AfterAll` devono essere `static`

I metodi annotati con:

- `@BeforeAll`
- `@AfterAll`

devono essere `static` (nel comportamento standard di JUnit 5) perché:

- `@BeforeAll` **viene eseguito prima che venga istanziata la classe di test**
- `@AfterAll` **viene eseguito dopo che l'ultima istanza della classe di test è stata distrutta**

Poiché in quel momento **non esiste un oggetto della classe**, il metodo deve appartenere alla classe stessa, quindi essere `static`.

Se non vengono dichiarati `static`, si ottiene un **errore a runtime**.

Istanziazione della classe di test

JUnit adotta, per impostazione predefinita, il seguente comportamento:

- **La classe di test viene istanziata una volta per ogni metodo annotato con** `@Test`
- **Dopo l'esecuzione del test, l'istanza viene distrutta**

Questo significa che:

- **Se ho 5 metodi** `@Test`**, la classe verrà creata e distrutta 5 volte**
- **Ogni test lavora su un'istanza diversa**
- **I test sono indipendenti tra loro**

Questo meccanismo è fondamentale per evitare effetti collaterali tra test.

Ritorno dei metodi del ciclo di vita

I metodi annotati con:

- `@BeforeAll`
- `@AfterAll`
- `@BeforeEach`
- `@AfterEach`

non dovrebbero prevedere un valore di ritorno.

Formalmente, anche se venisse dichiarato un tipo di ritorno diverso da `void`:

- **Non verrebbe generato un errore**
- **Il valore restituito verrebbe semplicemente ignorato dal framework**

Il comportamento corretto è quindi dichiararli `void`.

Segnalare un fallimento

I metodi del ciclo di vita possono segnalare un errore:

- Sollevando un'**eccezione unchecked** (ad esempio `RuntimeException`)

In tal caso:

- Il test (o l'intera suite) viene considerato fallito
- L'esecuzione può essere interrotta

Note su `@BeforeEach`

Annotare un metodo con `@BeforeEach` è, dal punto di vista funzionale, spesso equivalente a inizializzare direttamente un attributo nel momento della dichiarazione.

Esempio:

```
public class RubricaMock{
    private Rubrica rubrica;
    @BeforeEach
    void setup(){
        rubrica = new Rubrica(1, "Giorgia", 2020, 2);
```

```
    }
}
```

oppure:

```
private MyClass obj;

@BeforeEach
void setup() {
    obj = new MyClass();
}
```

Dal punto di vista progettuale, l'uso di `@BeforeEach` è generalmente preferibile perché:

- **rende esplicita la fase di inizializzazione;**
- **migliora la leggibilità della classe di test;**
- **separa chiaramente la configurazione iniziale dalla logica dei test;**
- **facilita l'evoluzione verso test di integrazione o configurazioni più complesse.**

L'inizializzazione non è "nascosta" nella dichiarazione del campo, ma diventa una fase strutturata del ciclo di vita del test.

Comportamento operativo

Con `@BeforeEach`:

- L'attributo viene dichiarato nella classe
- L'inizializzazione viene eseguita esplicitamente prima di ogni metodo `@Test`

Questo garantisce che:

- ogni test parta da uno stato iniziale noto e controllato;
- non vi siano dipendenze implicite tra test diversi;
- la configurazione sia centralizzata e facilmente modificabile.